

LAPORAN PRAKTIKUM

STRUKTUR DATA

Dosen Pengampu : Dr. Wahyudi, S.T, M.T



Disusun Oleh:
Annafi Al Ghifari
2511533013

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan tugas mata kuliah *Struktur Data* ini dengan baik dan tepat waktu.

Laporan/tugas ini disusun sebagai salah satu bentuk pemenuhan kewajiban akademik pada mata kuliah Struktur Data. Dalam penyusunannya, penulis berusaha memahami konsep-konsep dasar yang berkaitan dengan struktur data, seperti pengelolaan data menggunakan array, stack, queue, serta implementasi dalam pemrograman. Diharapkan melalui tugas ini, penulis dapat meningkatkan pemahaman serta kemampuan dalam menerapkan konsep struktur data secara praktis.

Penulis menyadari bahwa dalam penyusunan tugas ini masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data serta semua pihak yang telah membantu dalam proses penyusunan tugas ini.

Semoga laporan ini dapat memberikan manfaat bagi penulis khususnya dan bagi pembaca pada umumnya.

Padang, 21 Mei 2026

Annafi Al Ghifari

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
BAB II PEMBAHASAN.....	2
2.3 Program Dan Kodingan	2
2. Tugas.....	21
BAB III KESIMPULAN	33
3.1 Ringkasan.....	33
3.2 Saran	33
DAFTAR PUSTAKA	34

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia komputer dan teknologi informasi, pengolahan data merupakan hal yang sangat penting. Data yang tersusun dengan rapi dan teratur akan memudahkan proses pencarian, pengelompokan, serta analisis informasi. Oleh karena itu, diperlukan suatu metode atau algoritma yang dapat digunakan untuk mengurutkan data secara efektif dan efisien. Proses pengurutan data tersebut dikenal dengan istilah sorting.

Algoritma sorting merupakan salah satu materi dasar dalam pemrograman dan struktur data yang wajib dipahami oleh mahasiswa informatika. Dengan memahami algoritma pengurutan, seseorang dapat meningkatkan kemampuan dalam menyusun logika program serta menyelesaikan berbagai permasalahan yang berkaitan dengan pengolahan data. Terdapat berbagai jenis algoritma pengurutan, namun beberapa metode dasar yang paling sering dipelajari adalah Insertion Sort, Selection Sort, dan Bubble Sort.

Insertion Sort adalah metode pengurutan dengan cara menyisipkan data ke posisi yang sesuai pada bagian data yang telah teratur. Selection Sort bekerja dengan memilih nilai terkecil atau terbesar dari sekumpulan data untuk ditempatkan pada posisi yang benar. Sementara itu, Bubble Sort melakukan pengurutan dengan membandingkan dua data yang berdekatan secara berulang hingga data tersusun dengan benar. Ketiga metode tersebut memiliki cara kerja, tingkat efisiensi, serta kelebihan dan kekurangan yang berbeda.

BAB II

PEMBAHASAN

2.1 Program Dan Kodingan

1. MergeSort_2511533013

```
2. package pekan8_2511533013;
3. public class MergeSort_2511533013 {
4.     void merge(int arr[], int l_3013, int m_3013, int r_3013) {
5.         // Find sizes of two subarrays to be merged
6.         int n1_3013 = m_3013 - l_3013 + 1;
7.         int n2_3013 = r_3013 - m_3013;
8.         /* Create temp arrays */
9.         int L_3013[] = new int[n1_3013];
10.        int R_3013[] = new int[n2_3013];
11.        /* Copy data to temp arrays */
12.        for (int i_3013 = 0; i_3013 < n1_3013; ++i_3013)
13.            L_3013[i_3013] = arr[l_3013 + i_3013];
14.        for (int j_3013 = 0; j_3013 < n2_3013; ++j_3013)
15.            R_3013[j_3013] = arr[m_3013 + 1 + j_3013];
16.        int i_3013 = 0, j_3013 = 0;
17.        // Initial index of merged subarray array
18.        int k_3013 = l_3013;
19.        while (i_3013 < n1_3013 && j_3013 < n2_3013) {
20.            if (L_3013[i_3013] <= R_3013[j_3013]) {
21.                arr[k_3013] = L_3013[i_3013];
22.                i_3013++;
23.            } else {
24.                arr[k_3013] = R_3013[j_3013];
25.                j_3013++;
26.            }
27.            k_3013++;
28.        }
29.        /* Copy remaining elements of L[] if any */
30.        while (i_3013 < n1_3013) {
31.            arr[k_3013] = L_3013[i_3013];
32.            i_3013++;
33.            k_3013++;
34.        }
35.        /* Copy remaining elements of R[] if any */
36.        while (j_3013 < n2_3013) {
37.            arr[k_3013] = R_3013[j_3013];
38.            j_3013++;
39.            k_3013++;
40.        }
41.    }
```

```

42.     void sort(int arr_3013[], int l_3013, int r_3013) {
43.         if (l_3013 < r_3013) {
44.             // Find the middle point
45.             int m_3013 = (l_3013 + r_3013) / 2;
46.             // Sort first and second halves
47.             sort(arr_3013, l_3013, m_3013);
48.             sort(arr_3013, m_3013 + 1, r_3013);
49.             // Merge the sorted halves
50.             merge(arr_3013, l_3013, m_3013, r_3013);
51.         }
52.     }
53.     /* A utility function to print array of size n */
54.     static void printArray(int arr_3013[]) {
55.         int n_3013 = arr_3013.length;
56.         for (int i_3013 = 0; i_3013 < n_3013; ++i_3013)
57.             System.out.print(arr_3013[i_3013] + " ");
58.         System.out.println();
59.     }
60.     public static void main(String args[]) {
61.         int arr_3013[] = { 12, 11, 13, 5, 6, 7 };
62.         System.out.println("Sebelum terurut");
63.         printArray(arr_3013);
64.         MergeSort_2511533013 ob_3013 = new MergeSort_2511533013();
65.         ob_3013.sort(arr_3013, 0, arr_3013.length - 1);
66.         System.out.println("\nSesudah Terurut menggunakan merge
Sort");
67.         printArray(arr_3013);
68.     }
69. }
70.

```

OUTPUT:

```

Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13

```

Penjelasan :

1. **Class MergeSort_2511533013**

Class ini digunakan untuk mengurutkan data menggunakan algoritma Merge Sort.

2. **Method merge()**

Method ini digunakan untuk menggabungkan dua subarray yang sudah terurut menjadi satu array terurut.

3. **Variabel n1_3013**

Digunakan untuk menyimpan jumlah elemen subarray kiri.

4. **Variabel n2_3013**

Digunakan untuk menyimpan jumlah elemen subarray kanan.

5. **Array L_3013[] dan R_3013[]**

Digunakan sebagai array sementara untuk menyimpan data bagian kiri dan kanan.

6. **Perulangan For pertama**

Digunakan untuk menyalin data dari array utama ke array kiri L_3013.

7. **Perulangan For kedua**

Digunakan untuk menyalin data dari array utama ke array kanan R_3013.

8. **Variabel i_3013 dan j_3013**

Digunakan sebagai indeks untuk array kiri dan kanan.

9. **Variabel k_3013**

Digunakan sebagai indeks array utama saat proses penggabungan.

10. **Perulangan While pertama**

Digunakan untuk membandingkan elemen array kiri dan kanan.

11. **Kondisi L_3013[i_3013] <= R_3013[j_3013]**

Digunakan untuk menentukan elemen yang lebih kecil.

12. **arr[k_3013] = L_3013[i_3013]**

Digunakan untuk memasukkan elemen kiri ke array utama.

13. `arr[k_3013] = R_3013[j_3013]`

Digunakan untuk memasukkan elemen kanan ke array utama.

14. Perulangan While kedua

Digunakan untuk menyalin sisa elemen array kiri jika masih ada.

15. Perulangan While ketiga

Digunakan untuk menyalin sisa elemen array kanan jika masih ada.

16. Method `sort()`

Method ini digunakan untuk membagi array menjadi beberapa bagian secara rekursif.

17. Kondisi `if (l_3013 < r_3013)`

Digunakan untuk memastikan array masih bisa dibagi.

18. Variabel `m_3013`

Digunakan untuk menentukan titik tengah array.

19. `sort(arr_3013, l_3013, m_3013)`

Digunakan untuk mengurutkan bagian kiri array.

20. `sort(arr_3013, m_3013 + 1, r_3013)`

Digunakan untuk mengurutkan bagian kanan array.

21. `merge(arr_3013, l_3013, m_3013, r_3013)`

Digunakan untuk menggabungkan dua bagian array yang sudah terurut.

22. Method `printArray()`

Digunakan untuk menampilkan isi array ke layar.

23. Variabel `n_3013` pada `printArray()`

Digunakan untuk menyimpan jumlah elemen array.

24. Perulangan For pada `printArray()`

Digunakan untuk mencetak seluruh elemen array.

25. Method `main`

Merupakan titik awal eksekusi program.

26. Variabel `arr_3013`

Digunakan untuk menyimpan data yang akan diurutkan.

27. Object ob_3013

Digunakan untuk memanggil method sort() dari class

MergeSort_2511533013.

28. ob_3013.sort(arr_3013, 0, arr_3013.length - 1)

Digunakan untuk menjalankan proses Merge Sort pada seluruh array.

29. Output “Sebelum terurut” dan “Sesudah Terurut menggunakan Merge Sort”

Digunakan untuk menampilkan data sebelum dan sesudah proses sorting dilakukan.

2. QuickSort_2511533013

```
3. package pekan8_2511533013;
4.
5. public class QuickSort_2511533013 {
6.     static void swap (int[] arr, int i_3013, int j_3013) {
7.         int temp = arr[i_3013];
8.         arr[i_3013] = arr[j_3013];
9.         arr[j_3013] = temp;
10.    }
11.    // metode tambahan untuk mengatur pivot pakai median of three
12.    static void medianofThree(int[] arr, int low, int high) {
13.        int mid = low + (high - low) / 2;
14.
15.        // urutkan elemen low, mid, dan high
16.        if (arr[low] > arr[mid]) {
17.            swap(arr, low, mid);
18.        }
19.        if (arr[low] > arr[high]) {
20.            swap(arr, low, high);
21.        }
22.        if (arr[mid] > arr[high]) {
23.            swap(arr, mid, high);
24.        }
25.        swap(arr, mid, high);
26.    }
27.    static int partition(int[] arr, int low, int high)
28.    {
29.        // Panggil fungsi medianofthree sebelum menentukan
        pivot
30.        medianofThree(arr, low, high);
31.
```

```

32.         int pivot = arr[high]; // sekarang arr[high] berisi
           nilai median
33.         int i = (low - 1);
34.
35.         for (int j = low; j < high; j++) {
36.             // Jika elemen saat ini lebih kecil dari atau sama
           dengan pivot
37.             if (arr[j] <= pivot) {
38.                 // Increment indeks elemen yg lebih kecil
39.                 i++;
40.                 swap(arr, i, j);
41.             }
42.         }
43.         swap(arr, i + 1, high);
44.         return i + 1;
45.     }
46.     static void quickSort(int[] arr, int low, int high) {
47.         if (low < high) {
48.             int pi = partition(arr, low, high);
49.
50.             // Secara rekursif sort elemen sebelum dan sesudah
           partisi
51.             quickSort(arr, low, pi - 1);
52.             quickSort(arr, pi + 1, high);
53.         }
54.     }
55.     public static void printArr(int[] arr) {
56.         for (int i = 0; i < arr.length; i++) {
57.             System.out.print(arr[i] + " ");
58.         }
59.         System.out.println();
60.     }
61.     public static void main(String[] args) {
62.         int[] arr = {10, 7, 8, 9, 1, 5};
63.         int n = arr.length;
64.
65.         System.out.println("Data sebelum diurutkan:");
66.         printArr(arr);
67.
68.         quickSort(arr, 0, n - 1);
69.
70.         System.out.println("Data setelah diurutkan:");
71.         printArr(arr);
72.     }
73. }
74.

```

OUTPUT:

```
Data sebelum diurutkan:
10 7 8 9 1 5
Data setelah diurutkan:
1 5 7 8 9 10
```

Penjelasan:

1. **Class QuickSort_2511533013**
Class ini digunakan untuk mengurutkan data menggunakan algoritma Quick Sort.
2. **Method swap()**
Method ini digunakan untuk menukar posisi dua elemen dalam array.
3. **Variabel temp**
Digunakan sebagai variabel sementara saat proses pertukaran data.
4. **Method medianofThree()**
Method ini digunakan untuk menentukan pivot menggunakan metode median of three.
5. **Variabel mid**
Digunakan untuk mencari indeks tengah array.
6. **Kondisi If pada medianofThree()**
Digunakan untuk mengurutkan elemen low, mid, dan high agar didapat nilai median.
7. **Method swap(arr, mid, high)**
Digunakan untuk memindahkan nilai median ke posisi akhir array sebagai pivot.

8. Method partition()

Method ini digunakan untuk membagi array berdasarkan pivot.

9. Pemanggilan medianofThree()

Digunakan untuk menentukan pivot sebelum proses partisi dilakukan.

10. Variabel pivot

Digunakan untuk menyimpan nilai pivot dari array.

11. Variabel i

Digunakan untuk menyimpan indeks elemen yang lebih kecil dari pivot.

12. Perulangan For pada partition()

Digunakan untuk membandingkan setiap elemen array dengan pivot.

13. Kondisi If pada partition()

Digunakan untuk mengecek apakah elemen lebih kecil atau sama dengan pivot.

14. Proses swap(arr, i, j)

Digunakan untuk menukar elemen yang lebih kecil ke bagian kiri array.

15. swap(arr, i + 1, high)

Digunakan untuk menempatkan pivot pada posisi yang benar.

16. return i + 1

Digunakan untuk mengembalikan indeks posisi pivot.

17. Method quickSort()

Method ini digunakan untuk menjalankan proses Quick Sort secara rekursif.

18. Kondisi if (low < high)

Digunakan untuk memastikan proses sorting masih dapat dilakukan.

19. Variabel pi

Digunakan untuk menyimpan indeks partisi.

20. quickSort(arr, low, pi - 1)

Digunakan untuk mengurutkan bagian kiri pivot.

21. **quickSort(arr, pi + 1, high)**

Digunakan untuk mengurutkan bagian kanan pivot.

22. **Method printArr()**

Digunakan untuk menampilkan isi array ke layar.

23. **Perulangan For pada printArr()**

Digunakan untuk mencetak seluruh elemen array.

24. **Method main**

Merupakan titik awal eksekusi program.

25. **Variabel arr**

Digunakan untuk menyimpan data yang akan diurutkan.

26. **Variabel n**

Digunakan untuk menyimpan jumlah elemen array.

27. **printArr(arr)**

Digunakan untuk menampilkan data sebelum dan sesudah sorting.

28. **quickSort(arr, 0, n - 1)**

Digunakan untuk menjalankan proses Quick Sort pada seluruh array.

3. ShellSort_2511533013

```
4. package pekan8_2511533013;
5.
6. public class ShellSort_2511533013 {
7.     public static void shellSort(int[] A_3013) {
8.         int n_3013 = A_3013.length;
9.         int gap_3013 = n_3013 / 2;
10.        while (gap_3013 > 0) {
11.            for (int i_3013 = gap_3013; i_3013 < n_3013;
12.                i_3013++) {
13.                int temp_3013 = A_3013[i_3013];
14.                int j_3013 = i_3013;
15.                while (j_3013 >= gap_3013 &&
16.                    A_3013[j_3013 - gap_3013] > temp_3013) {
17.                    A_3013[j_3013] = A_3013[j_3013 -
18.                        gap_3013];
19.                    j_3013 = j_3013 - gap_3013;
```

```

17.         }
18.         A_3013[j_3013] = temp_3013;
19.     }
20.     gap_3013 = gap_3013 / 2;
21. }
22. }
23. public static void main(String[] args) {
24.     int[] data_3013 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
25.     System.out.print("Sebelum: ");
26.     printArray(data_3013);
27.     shellSort(data_3013);
28.     System.out.print("Sesudah (Shell Sort): ");
29.     printArray(data_3013);
30. }
31. public static void printArray(int[] arr_3013) {
32.     for (int i_3013 : arr_3013) System.out.print(i_3013 +
33. " ");
34.     System.out.println();
35. }
36.

```

OUTPUT:

```

Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10

```

Penjelasan:

1. **Class ShellSort_2511533013**
Class ini digunakan untuk mengurutkan data menggunakan algoritma Shell Sort.
2. **Method shellSort()**
Method ini digunakan untuk menjalankan proses pengurutan Shell Sort pada array.
3. **Variabel n_3013**
Digunakan untuk menyimpan jumlah elemen array menggunakan `A_3013.length`.
4. **Variabel gap_3013**
Digunakan untuk menentukan jarak antar elemen yang dibandingkan.

5. **Perulangan While pertama**
Digunakan untuk menjalankan proses sorting selama nilai gap masih lebih besar dari 0.
6. **Perulangan For**
Digunakan untuk melakukan proses insertion sort berdasarkan gap tertentu.
7. **Variabel temp_3013**
Digunakan untuk menyimpan elemen yang akan dipindahkan sementara.
8. **Variabel j_3013**
Digunakan sebagai indeks untuk proses pergeseran elemen.
9. **Perulangan While kedua**
Digunakan untuk menggeser elemen yang lebih besar dari temp_3013.
10. **Kondisi a_3013[j_3013 - gap_3013] > temp_3013**
Digunakan untuk mengecek apakah elemen sebelumnya lebih besar dari elemen yang sedang dibandingkan.
11. **Proses Pergeseran Data**
Digunakan untuk memindahkan elemen ke posisi kanan sesuai nilai gap.
12. **j_3013 = j_3013 - gap_3013**
Digunakan untuk memindahkan indeks ke posisi sebelumnya berdasarkan gap.
13. **a_3013[j_3013] = temp_3013**
Digunakan untuk menempatkan elemen pada posisi yang benar.
14. **gap_3013 = gap_3013 / 2**
Digunakan untuk memperkecil nilai gap setiap iterasi.
15. **Method main**
Merupakan titik awal eksekusi program.
16. **Variabel data_3013**
Digunakan untuk menyimpan data yang akan diurutkan.
17. **Method printArray()**
Digunakan untuk menampilkan isi array ke layar.
18. **shellSort(data_3013)**
Digunakan untuk menjalankan proses Shell Sort pada array.
19. **Perulangan For Each pada printArray()**
Digunakan untuk mencetak seluruh elemen array satu per satu.
20. **Output “Sebelum” dan “Sesudah”**
Digunakan untuk menampilkan data sebelum dan setelah proses sorting dilakukan.

4. BubbleSortGUI_2511533013

```
2. package pekan8_2511533013;
3.
4. import java.awt.BorderLayout;
5. import java.awt.Color;
6. import java.awt.Dimension;
7. import java.awt.FlowLayout;
8. import java.awt.Font;
9.
10. import javax.swing.BorderFactory;
11. import javax.swing.JButton;
12. import javax.swing.JFrame;
13. import javax.swing.JLabel;
14. import javax.swing.JOptionPane;
15. import javax.swing.JPanel;
16. import javax.swing.JScrollPane;
17. import javax.swing.JTextArea;
18. import javax.swing.JTextField;
19. import javax.swing.SwingConstants;
20. import javax.swing.SwingUtilities;
21.
22. public class BubbleSortGUI_2511533013 extends JFrame {
23.     private static final long serialVersionUID = 1L;
24.     private int[] array_3013;
25.     private JLabel[] labelArray_3013;
26.     private JButton stepButton_3013, resetButton_3013,
        setButton_3013;
27.     private JTextField inputField_3013;
28.     private JPanel panelArray_3013;
29.     private JTextArea stepArea_3013;
30.
31.     private int i_3013 = 1, j_3013;
32.     private boolean sorting_3013 = false;
33.     private int stepCount_3013 = 1;
34.
35.
36.     /**
37.      * Create the frame.
38.      */
39.     public BubbleSortGUI_2511533013() {
40.         setTitle("Bubble Sort Langkah per Langkah");
41.         setSize(772, 400);
42.         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
43.         setLocationRelativeTo(null);
44.         getContentPane().setLayout(new BorderLayout());
45.
46.         // Panel input
47.         JPanel inputPanel_3013 = new JPanel(new FlowLayout());
48.         inputField_3013 = new JTextField(30);
49.         setButton_3013 = new JButton("Set Array");
50.         inputPanel_3013.add(new JLabel("Masukkan angka (pisahkan
            dengan koma):"));
```



```

51.     inputPanel_3013.add(inputField_3013);
52.     inputPanel_3013.add(setButton_3013);
53.
54.     // Panel array visual
55.     panelArray_3013 = new JPanel();
56.     panelArray_3013.setLayout(new FlowLayout());
57.
58.     // Panel kontrol
59.     JPanel controlPanel_3013 = new JPanel();
60.     stepButton_3013 = new JButton("Langkah Selanjutnya");
61.     resetButton_3013 = new JButton("Reset");
62.     stepButton_3013.setEnabled(false);
63.     controlPanel_3013.add(stepButton_3013);
64.     controlPanel_3013.add(resetButton_3013);
65.     // Area teks untuk log langkah-langkah
66.     stepArea_3013 = new JTextArea(8, 60);
67.     stepArea_3013.setEditable(false);
68.     stepArea_3013.setFont(new Font("Monospaced", Font.PLAIN,
14));
69.     JScrollPane scrollPane_3013 = new
JScrollPane(stepArea_3013);
70.
71.     // Tambahkan panel ke frame
72.     getContentPane().add(inputPanel_3013,
BorderLayout.NORTH);
73.     getContentPane().add(panelArray_3013,
BorderLayout.CENTER);
74.     getContentPane().add(controlPanel_3013,
BorderLayout.SOUTH);
75.     getContentPane().add(scrollPane_3013,
BorderLayout.EAST);
76.
77.     // Event Set Array
78.     setButton_3013.addActionListener(e ->
setArrayFromInput());
79.
80.     // Event Langkah Selanjutnya
81.     stepButton_3013.addActionListener(e -> performStep());
82.
83.     // Event Reset
84.     resetButton_3013.addActionListener(e -> reset());
85. }
86. private void setArrayFromInput() {
87.     String text_3013 = inputField_3013.getText().trim();
88.     if (text_3013.isEmpty()) return;
89.     String[] parts_3013 = text_3013.split(",");
90.     array_3013 = new int[parts_3013.length];
91.     try {
92.         for (int k_3013 = 0; k_3013 < parts_3013.length;
k_3013++) {
93.             array_3013[k_3013] =
Integer.parseInt(parts_3013[k_3013].trim());
94.         }

```

```

95.         } catch (NumberFormatException e) {
96.             JOptionPane.showMessageDialog(this, "Masukkan hanya
angka "
97.                 + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
98.             return;
99.         }
100.         i_3013 = 0;
101.         j_3013 = 0;
102.         stepCount_3013 = 1;
103.         sorting_3013 = true;
104.         stepButton_3013.setEnabled(true);
105.         stepArea_3013.setText("");
106.         panelArray_3013.removeAll();
107.         labelArray_3013 = new JLabel[array_3013.length];
108.         for (int k_3013 = 0; k_3013 < array_3013.length;
k_3013++) {
109.             labelArray_3013[k_3013] = new
JLabel(String.valueOf(array_3013[k_3013]));
110.             labelArray_3013[k_3013].setFont(new
Font("Arial", Font.BOLD, 24));
111.             labelArray_3013[k_3013].setOpaque(true);
112.             labelArray_3013[k_3013].setBackground(Color.WHITE);
113.             labelArray_3013[k_3013].setBorder(BorderFactory.createLineBorde
r(Color.BLACK));
114.             labelArray_3013[k_3013].setPreferredSize(new
Dimension(50, 50));
115.             labelArray_3013[k_3013].setHorizontalAlignment(SwingConstants.C
ENTER);
116.             panelArray_3013.add(labelArray_3013[k_3013]);
117.         }
118.
119.         panelArray_3013.revalidate();
120.         panelArray_3013.repaint();
121.     }
122.     private void performStep() {
123.         if (!sorting_3013 || i_3013 >= array_3013.length
- 1) {
124.             sorting_3013 = false;
125.             stepButton_3013.setEnabled(false);
126.             JOptionPane.showMessageDialog(this, "Sorting
selesai!");
127.             return;
128.         }
129.         resetHighlights();
130.         StringBuilder stepLog_3013 = new StringBuilder();
131.         labelArray_3013[j_3013].setBackground(Color.CYAN);
132.         labelArray_3013[j_3013 +
1].setBackground(Color.CYAN);

```

```

133.         if (array_3013[j_3013] > array_3013[j_3013 + 1])
134.         {
135.             // Swap
136.             int temp_3013 = array_3013[j_3013];
137.             array_3013[j_3013] = array_3013[j_3013 + 1];
138.             array_3013[j_3013 + 1] = temp_3013;
139.             labelArray_3013[j_3013].setBackground(Color.RED);
140.             labelArray_3013[j_3013 +
141. 1].setBackground(Color.RED);
142.             stepLog_3013.append("Langkah
143. ").append(stepCount_3013).append(": Menukar elemen ke-")
144.             .append(j_3013).append("
145. ") .append(array_3013[j_3013 + 1]).append(" dengan ke-")
146.             .append(j_3013 + 1).append("
147. ") .append(array_3013[j_3013]).append("\n");
148.         } else {
149.             stepLog_3013.append("Langkah
150. ").append(stepCount_3013).append(": Tidak ada pertukaran antara
151. ke-")
152.             .append(j_3013).append(" dan ke-")
153.             .append(j_3013 + 1).append("\n");
154.         }
155.         stepLog_3013.append("Hasil:
156. ").append(arrayToString(array_3013)).append("\n\n");
157.         stepArea_3013.append(stepLog_3013.toString());
158.         updateLabels();
159.         j_3013++;
160.         if (j_3013 >= array_3013.length - i_3013 - 1) {
161.             j_3013 = 0;
162.             i_3013++;
163.         }
164.         stepCount_3013++;
165.         if (i_3013 >= array_3013.length - 1) {
166.             sorting_3013 = false;
167.             stepButton_3013.setEnabled(false);
168.             JOptionPane.showMessageDialog(this, "Sorting
169. selesai!");
170.         }
171.     }
172.     private void updateLabels() {
173.         for (int k_3013 = 0; k_3013 < array_3013.length;
174. k_3013++) {
175.             labelArray_3013[k_3013].setText(String.valueOf(array_3013[k_3013]));
176.         }
177.     }
178.     private void resetHighlights() {
179.         for (JLabel label_3013 : labelArray_3013) {
180.             label_3013.setBackground(Color.WHITE);
181.         }
182.     }

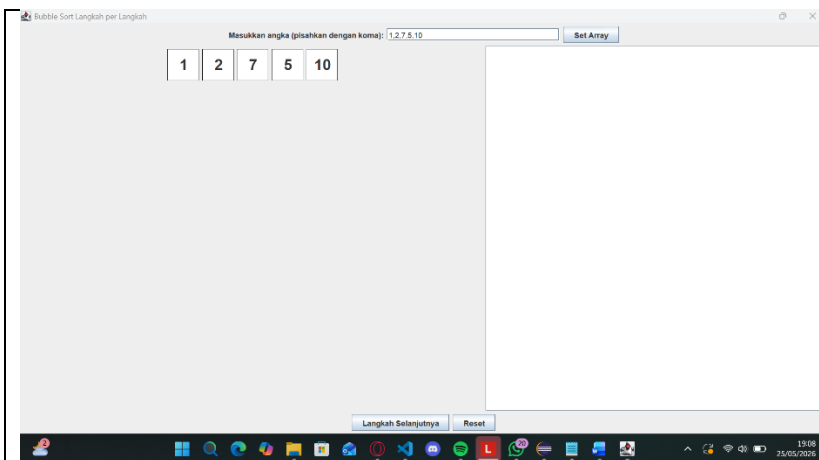
```

```

172.     }
173.
174.     private void reset() {
175.         inputField_3013.setText("");
176.         panelArray_3013.removeAll();
177.         panelArray_3013.revalidate();
178.         panelArray_3013.repaint();
179.         stepArea_3013.setText("");
180.         stepButton_3013.setEnabled(false);
181.         sorting_3013 = false;
182.         i_3013 = 0;
183.         j_3013 = 0;
184.         stepCount_3013 = 1;
185.     }
186.
187.     private String arrayToString(int[] arr_3013) {
188.         StringBuilder sb_3013 = new StringBuilder();
189.         for (int k_3013 = 0; k_3013 < arr_3013.length;
190.             k_3013++) {
191.             sb_3013.append(arr_3013[k_3013]);
192.             if (k_3013 < arr_3013.length - 1)
193.                 sb_3013.append(", ");
194.         }
195.         return sb_3013.toString();
196.     }
197.     public static void main(String[] args) {
198.         SwingUtilities.invokeLater(() -> {
199.             BubbleSortGUI_2511533013 gui_3013 = new
200.             BubbleSortGUI_2511533013();
201.             gui_3013.setVisible(true);
202.         });
203.     }

```

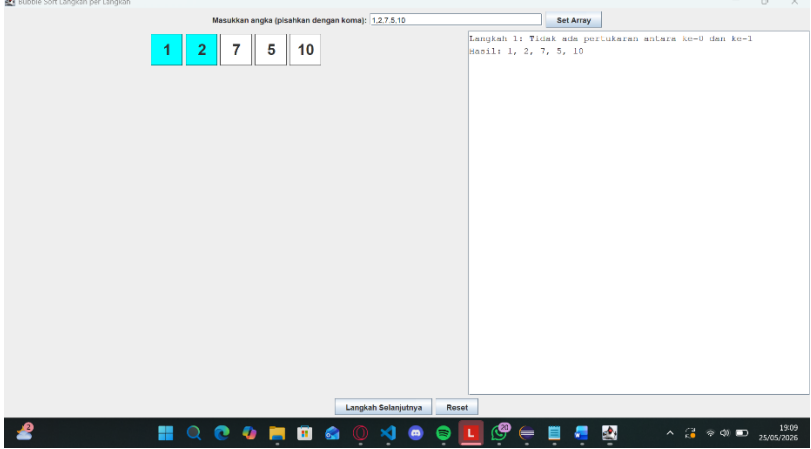
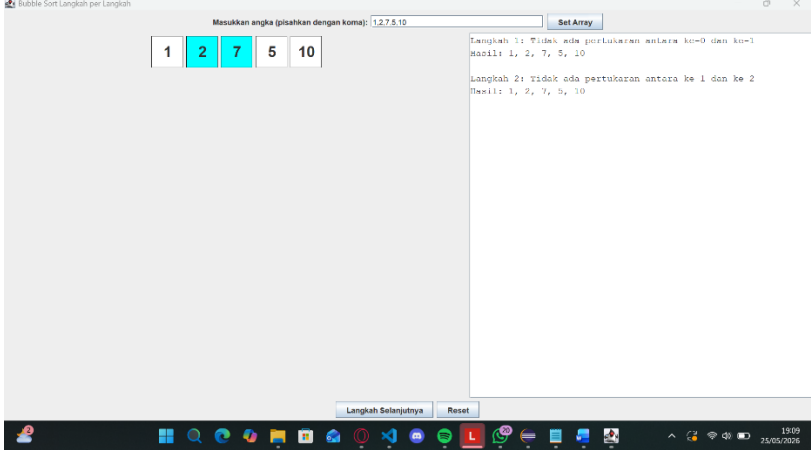
OUTPUT:

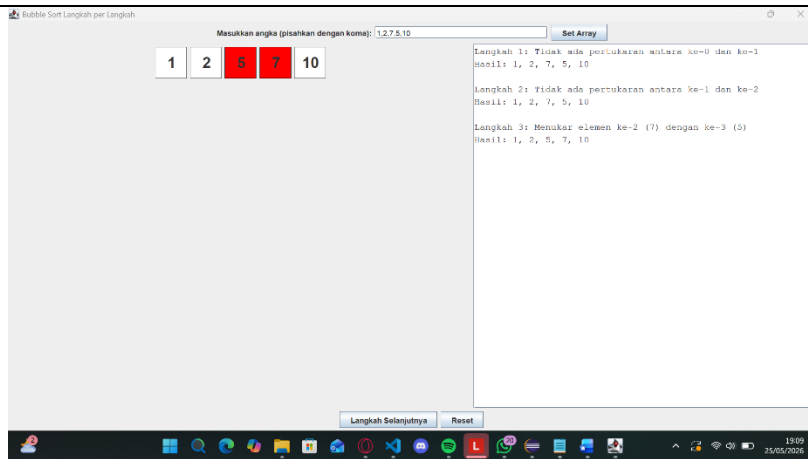


kemudian menekan tombol **Set Array**.

Program menampilkan data array dalam bentuk kotak menggunakan JLabel.

Pada tahap ini proses Bubble Sort belum berjalan sehingga semua

	kotak masih berwarna putih dan area langkah masih kosong.
	Tekan tombol Langkah Selanjutnya. Program membandingkan elemen indeks ke-0 dan ke-1 (1 dan 2) Karena $1 < 2$ maka tidak terjadi pertukaran data.
	Program membandingkan elemen indeks ke-1 dan ke-2 yaitu: 2 dan 7 Karena $2 < 7$ maka tidak terjadi pertukaran data. Kotak angka 2 dan 7 diberi warna biru karena sedang dibandingkan. Pada area log muncul: Langkah 2: Tidak ada pertukaran antara ke-1 dan ke-2 Hasil: 1, 2, 7, 5, 10



Pada gambar tersebut, program sedang menjalankan proses

Bubble Sort pada data:

1, 2, 7, 5, 10

Pada langkah ke-3, program membandingkan elemen indeks ke-2 dan ke-3 yaitu angka:

7 dan 5

Karena:

$7 > 5$

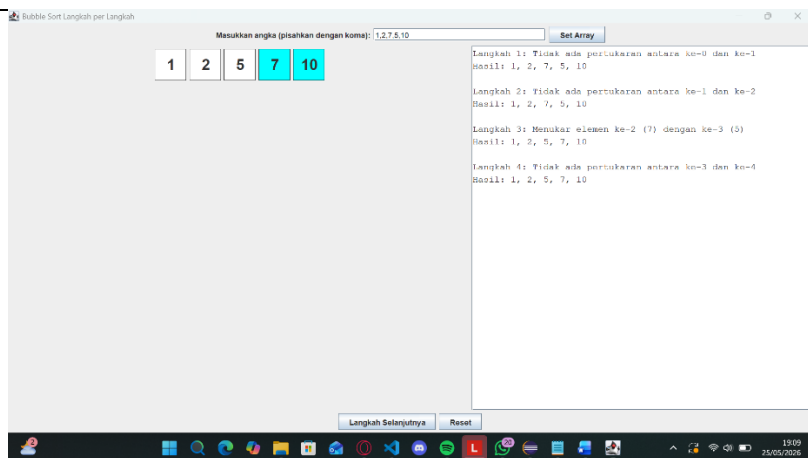
maka program melakukan proses pertukaran (swap).

Akibat pertukaran tersebut:

- angka 5 berpindah ke kiri
- angka 7 berpindah ke kanan

sehingga array berubah menjadi:

1, 2, 5, 7, 10



Pada awal proses:

Array ditampilkan sebagai kotak:

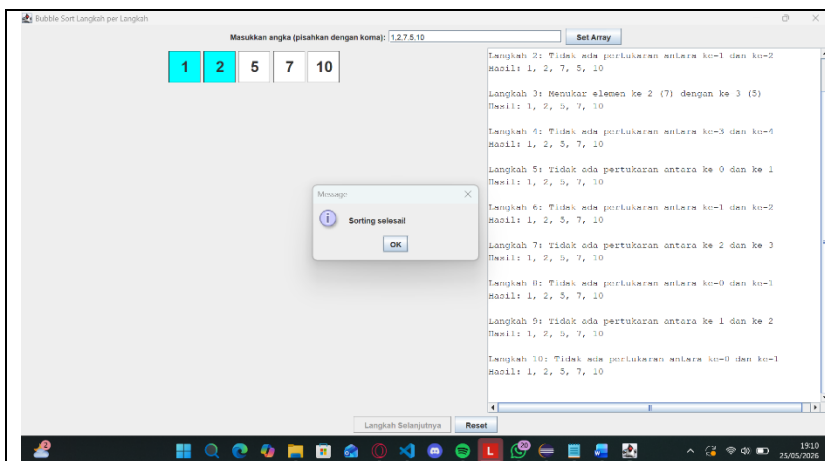
1, 2, 5, 7, 10

Warna biru muda menandakan elemen yang sedang dibandingkan.

Di panel kanan terdapat log proses Bubble Sort.

Contoh langkah yang terjadi:

	Perbandingan elemen ke-1 dan ke-2 - 1 dibandingkan dengan 2 - Karena $1 < 2$, tidak ada pertukaran. Perbandingan elemen ke-2 dan ke-3 - 2 dibandingkan dengan 7 - Tidak ada pertukaran. Perbandingan elemen ke-3 dan ke-4 - 7 dibandingkan dengan 5 - Karena $7 > 5$, dilakukan pertukaran. - Array berubah menjadi: 1, 2, 5, 7, 10 Perbandingan elemen ke-4 dan ke-5 - 7 dibandingkan dengan 10 - Tidak ada pertukaran. Jadi pada iterasi pertama, angka 7 “bergerak” ke kanan setelah ditukar dengan 5.
--	---



Pada gambar ini

Array sudah menjadi:

1, 2, 5, 7, 10

- Semua elemen sudahurut dari kecil ke besar.
- Program melanjutkan beberapa pengecekan untuk memastikan tidak ada lagi pertukaran.
- Karena tidak ada swap pada iterasi berikutnya, muncul pesan:

“Sorting selesai”

1. Tugas

4. Lagu_2511533013

```

2. package pekan8_2511533013;
3.
4. public class Lagu_2511533013 {
5.
6.     String judul_3013;
7.     String penyanyi_3013;
8.     int durasi_3013;
9.
10.    Lagu_2511533013(String judul_3013,
11.                     String penyanyi_3013,
12.                     int durasi_3013) {
13.
14.        this.judul_3013 = judul_3013;
15.        this.penyanyi_3013 = penyanyi_3013;
16.        this.durasi_3013 = durasi_3013;
17.    }
18.}
19.

```


OUTPUT:

Penjelasan :

1. Variabel

- judul_3013 → menyimpan judul lagu
- penyanyi_3013 → menyimpan nama penyanyi
- durasi_3013 → menyimpan durasi lagu dalam detik

2. Constructor

```
Lagu_2511533013(String judul_3013,  
                String penyanyi_3013,  
                int durasi_3013)
```

Constructor digunakan untuk mengisi data lagu saat object dibuat.

3. This

```
this.judul_3013 = judul_3013;  
this.penyanyi_3013 = penyanyi_3013;  
this.durasi_3013 = durasi_3013;
```

Keyword this digunakan untuk membedakan variabel atribut class dengan parameter constructor. Nilai parameter akan disimpan ke dalam atribut object.

4. Sorting_2511533013

```
3. package pekan8_2511533013;  
4.  
5. import java.util.Scanner;  
6.  
7. public class Sorting_2511533013 {  
8.  
9.     Lagu_2511533013[] dataLagu_3013 =  
10.         new Lagu_2511533013[20];  
11.  
12.     int jumlahData_3013 = 0;  
13.  
14.     Scanner input_3013 = new Scanner(System.in);  
15.
```

```

16. // ===== INPUT DATA =====
17. void inputData_3013() {
18.
19.     System.out.print("Masukkan jumlah lagu: ");
20.     int jumlah_3013 = input_3013.nextInt();
21.     input_3013.nextLine();
22.
23.     for (int i_3013 = 0; i_3013 < jumlah_3013; i_3013++)
24.     {
25.         System.out.println("\nData Lagu Ke" + (i_3013 +
26.         1));
27.
28.         System.out.print("Judul Lagu : ");
29.         String judul_3013 =
30.             input_3013.nextLine();
31.
32.         System.out.print("Nama Artis : ");
33.         String penyanyi_3013 =
34.             input_3013.nextLine();
35.
36.         System.out.print("Durasi Lagu (detik) : ");
37.         int durasi_3013 =
38.             input_3013.nextInt();
39.
40.         input_3013.nextLine();
41.
42.         dataLagu_3013[jumlahData_3013++] =
43.             new Lagu_2511533013(
44.                 judul_3013,
45.                 penyanyi_3013,
46.                 durasi_3013
47.             );
48.     }
49.
50. // ===== TAMPIL DATA =====
51. void tampilData_3013() {
52.
53.     for (int i_3013 = 0;
54.         i_3013 < jumlahData_3013;
55.         i_3013++) {
56.

```

```

57.         System.out.println(
58.             (i_3013 + 1) + ". "
59.             + dataLagu_3013[i_3013].judul_3013
60.             + " "
61.             + dataLagu_3013[i_3013].penyanyi_3013
62.             + " "
63.             + dataLagu_3013[i_3013].durasi_3013
64.             + " detik");
65.     }
66. }
67.
68. // ===== SHELL SORT =====
69. void shellSort_3013() {
70.
71.     for (int gap_3013 = jumlahData_3013 / 2;
72.          gap_3013 > 0;
73.          gap_3013 /= 2) {
74.
75.         for (int i_3013 = gap_3013;
76.              i_3013 < jumlahData_3013;
77.              i_3013++) {
78.
79.             Lagu_2511533013 temp_3013 =
80.                 dataLagu_3013[i_3013];
81.
82.             int j_3013;
83.
84.             for (j_3013 = i_3013;
85.                  j_3013 >= gap_3013
86.                  && dataLagu_3013[
87.                      j_3013 - gap_3013]
88.                      .judul_3013
89.                      .compareToIgnoreCase(
90.                          temp_3013.judul_3013) >
91.                          0;
92.                  j_3013 = gap_3013) {
93.
94.                 dataLagu_3013[j_3013] =
95.                     dataLagu_3013[
96.                         j_3013 - gap_3013];
97.
98.                 dataLagu_3013[j_3013] =

```

```

99.             temp_3013;
100.         }
101.     }
102. }
103.
104. // ===== QUICK SORT =====
105. int partition_3013(int low_3013,
106.                   int high_3013) {
107.
108.     int pivot_3013 =
109.         dataLagu_3013[high_3013]
110.             .durasi_3013;
111.
112.     int i_3013 = low_3013 - 1;
113.
114.     for (int j_3013 = low_3013;
115.          j_3013 < high_3013;
116.          j_3013++) {
117.
118.         if (dataLagu_3013[j_3013]
119.             .durasi_3013
120.             < pivot_3013) {
121.
122.             i_3013++;
123.
124.             Lagu_2511533013 temp_3013 =
125.                 dataLagu_3013[i_3013];
126.
127.             dataLagu_3013[i_3013] =
128.                 dataLagu_3013[j_3013];
129.
130.             dataLagu_3013[j_3013] =
131.                 temp_3013;
132.         }
133.     }
134.
135.     Lagu_2511533013 temp_3013 =
136.         dataLagu_3013[i_3013 + 1];
137.
138.     dataLagu_3013[i_3013 + 1] =
139.         dataLagu_3013[high_3013];
140.
141.     dataLagu_3013[high_3013] =

```

```

142.         temp_3013;
143.
144.         return i_3013 + 1;
145.     }
146.
147.     void quickSort_3013(int low_3013,
148.                         int high_3013) {
149.
150.         if (low_3013 < high_3013) {
151.
152.             int pi_3013 =
153.                 partition_3013(
154.                     low_3013,
155.                     high_3013);
156.
157.             quickSort_3013(
158.                 low_3013,
159.                 pi_3013 - 1);
160.
161.             quickSort_3013(
162.                 pi_3013 + 1,
163.                 high_3013);
164.         }
165.     }
166.
167.     // ===== MERGE SORT =====
168.     void merge_3013(int left_3013,
169.                    int mid_3013,
170.                    int right_3013) {
171.
172.         int n1_3013 =
173.             mid_3013 - left_3013 + 1;
174.
175.         int n2_3013 =
176.             right_3013 - mid_3013;
177.
178.         Lagu_2511533013[] L_3013 =
179.             new Lagu_2511533013[n1_3013];
180.
181.         Lagu_2511533013[] R_3013 =
182.             new Lagu_2511533013[n2_3013];
183.
184.         for (int i_3013 = 0;

```

```

185.         i_3013 < n1_3013;
186.         i_3013++) {
187.
188.             L_3013[i_3013] =
189.                 dataLagu_3013[
190.                     left_3013 + i_3013];
191.         }
192.
193.         for (int j_3013 = 0;
194.             j_3013 < n2_3013;
195.             j_3013++) {
196.
197.             R_3013[j_3013] =
198.                 dataLagu_3013[
199.                     mid_3013 + 1 + j_3013];
200.         }
201.
202.         int i_3013 = 0;
203.         int j_3013 = 0;
204.         int k_3013 = left_3013;
205.
206.         while (i_3013 < n1_3013
207.             && j_3013 < n2_3013) {
208.
209.             if (L_3013[i_3013]
210.                 .judul_3013
211.                 .compareToIgnoreCase(
212.                     R_3013[j_3013]
213.                     .judul_3013) <= 0) {
214.
215.                 dataLagu_3013[k_3013] =
216.                     L_3013[i_3013];
217.
218.                 i_3013++;
219.
220.             } else {
221.
222.                 dataLagu_3013[k_3013] =
223.                     R_3013[j_3013];
224.
225.                 j_3013++;
226.             }
227.

```

```

228.         k_3013++;
229.     }
230.
231.     while (i_3013 < n1_3013) {
232.
233.         dataLagu_3013[k_3013] =
234.             L_3013[i_3013];
235.
236.         i_3013++;
237.         k_3013++;
238.     }
239.
240.     while (j_3013 < n2_3013) {
241.
242.         dataLagu_3013[k_3013] =
243.             R_3013[j_3013];
244.
245.         j_3013++;
246.         k_3013++;
247.     }
248. }
249.
250. void mergeSort_3013(int left_3013,
251.                     int right_3013) {
252.
253.     if (left_3013 < right_3013) {
254.
255.         int mid_3013 =
256.             (left_3013 + right_3013) / 2;
257.
258.         mergeSort_3013(
259.             left_3013,
260.             mid_3013);
261.
262.         mergeSort_3013(
263.             mid_3013 + 1,
264.             right_3013);
265.
266.         merge_3013(
267.             left_3013,
268.             mid_3013,
269.             right_3013);
270.     }

```

```

271.     }
272.
273.     public static void main(String[] args) {
274.
275.         Sorting_2511533013 obj_3013 =
276.             new Sorting_2511533013();
277.
278.         System.out.println(
279.             "=== SORTING PLAYLIST LAGU ===");
280.
281.         obj_3013.inputData_3013();
282.
283.         int pilih_3013;
284.
285.         do {
286.
287.             System.out.println(
288.                 "\n=== MENU SORTING ===");
289.
290.             System.out.println(
291.                 "1. Shell Sort");
292.
293.             System.out.println(
294.                 "2. Quick Sort");
295.
296.             System.out.println(
297.                 "3. Merge Sort");
298.
299.             System.out.println(
300.                 "4. Exit");
301.
302.             System.out.print(
303.                 "Masukkan pilihan: ");
304.
305.             pilih_3013 =
306.                 obj_3013.input_3013.nextInt();
307.
308.             System.out.println(
309.                 "\nData Sebelum Sorting:");
310.
311.             obj_3013.tampilData_3013();
312.
313.             if (pilih_3013 == 1) {

```



```

314.
315.         obj_3013.shellSort_3013();
316.
317.         System.out.println(
318.             "\nData Setelah Shell Sort (Judul
319.             AZ):");
320.         obj_3013.tampilData_3013();
321.
322.     } else if (pilih_3013 == 2) {
323.
324.         obj_3013.quickSort_3013(
325.             0,
326.             obj_3013.jumlahData_3013 1);
327.
328.         System.out.println(
329.             "\nData Setelah Quick Sort
330.             (Durasi Asc):");
331.         obj_3013.tampilData_3013();
332.
333.     } else if (pilih_3013 == 3) {
334.
335.         obj_3013.mergeSort_3013(
336.             0,
337.             obj_3013.jumlahData_3013 1);
338.
339.         System.out.println(
340.             "\nData Setelah Merge Sort (Judul
341.             AZ):");
342.         obj_3013.tampilData_3013();
343.
344.     } else if (pilih_3013 == 4) {
345.
346.         System.out.println(
347.             "\nProgram selesai.");
348.
349.     } else {
350.
351.         System.out.println(
352.             "\nPilihan tidak valid!");
353.     }

```

```

354.
355.         } while (pilih_3013 != 4);
356.     }
357. }
358.

```

OUTPUT:

<pre> === SORTING PLAYLIST LAGU === Masukkan jumlah lagu: 3 Data Lagu Ke-1 Judul Lagu : la Nama Artis : li Durasi Lagu (detik) : 270 Data Lagu Ke-2 Judul Lagu : lu Nama Artis : le Durasi Lagu (detik) : 250 Data Lagu Ke-3 Judul Lagu : pp Nama Artis : pp Durasi Lagu (detik) : 190 </pre>		Masukkan jumlah lagu yang akan disimpan ke dalam playlist. Setelah itu, input data setiap lagu berupa judul lagu, nama artis, dan durasi detik. Data lagu kemudian disimpan ke dalam array object untuk diproses pada tahap sorting.
<pre> === MENU SORTING === 1. Shell Sort 2. Quick Sort 3. Merge Sort 4. Exit Masukkan pilihan: 1 Data Sebelum Sorting: 1. la - li - 270 detik 2. lu - le - 250 detik 3. pp - pp - 190 detik Data Setelah Shell Sort (Judul A-Z): 1. la - li - 270 detik 2. lu - le - 250 detik 3. pp - pp - 190 detik </pre>		User memilih menu Shell Sort. Program menampilkan data lagu sebelum sorting, kemudian melakukan pengurutan berdasarkan judul lagu secara alfabetis dari AZ menggunakan algoritma Shell Sort. Setelah proses selesai, data lagu tampil dalam urutan yang sudah terurut.
<pre> === MENU SORTING === 1. Shell Sort 2. Quick Sort 3. Merge Sort 4. Exit Masukkan pilihan: 2 Data Sebelum Sorting: 1. la - li - 270 detik 2. lu - le - 250 detik 3. pp - pp - 190 detik Data Setelah Quick Sort (Durasi Asc): 1. pp - pp - 190 detik 2. lu - le - 250 detik 3. la - li - 270 detik </pre>		Program menampilkan data sebelum sorting lalu mengurutkan data berdasarkan durasi lagu dari yang terkecil ke terbesar menggunakan algoritma Quick Sort. Hasil akhir menunjukkan lagu dengan durasi paling singkat berada di urutan pertama.

<pre>=== MENU SORTING === 1. Shell Sort 2. Quick Sort 3. Merge Sort 4. Exit Masukkan pilihan: 3 Data Sebelum Sorting: 1. pp - pp - 190 detik 2. lu - le - 250 detik 3. la - li - 270 detik Data Setelah Merge Sort (Judul A-Z): 1. la - li - 270 detik 2. lu - le - 250 detik 3. pp - pp - 190 detik</pre>	Program kembali menampilkan data sebelum sorting, kemudian melakukan pengurutan judul lagu secara alfabetis menggunakan algoritma Merge Sort. Setelah proses merge selesai, data lagu tampil dalam keadaan terurut AZ.
<pre>=== MENU SORTING === 1. Shell Sort 2. Quick Sort 3. Merge Sort 4. Exit Masukkan pilihan: 4 Data Sebelum Sorting: 1. la - li - 270 detik 2. lu - le - 250 detik 3. pp - pp - 190 detik Program selesai. PS C:\Users\annaf\git\repository\PrakSD2026_C_2511533013\src></pre>	User memilih menu Exit untuk keluar dari program. Program menampilkan pesan “Program selesai” yang menandakan seluruh proses input dan sorting telah berakhir.

BAB III

KESIMPULAN

3.1 Ringkasan

Berdasarkan hasil praktikum, program playlist lagu berhasil dibuat menggunakan bahasa Java dengan menerapkan algoritma Shell Sort, Quick Sort, dan Merge Sort. Program mampu menerima input data lagu, menyimpan data ke dalam array object, serta mengurutkan data berdasarkan judul lagu dan durasi lagu. Selain itu, program juga dapat menampilkan menu interaktif sehingga user dapat memilih metode sorting yang diinginkan sampai program selesai dijalankan.

3.2 Saran

Bagi pengajar, diharapkan dapat memberikan lebih banyak contoh kasus yang relevan dengan kehidupan nyata sehingga mahasiswa dapat lebih mudah memahami penerapan struktur data dalam dunia kerja.

DAFTAR PUSTAKA

1. Kadir, Abdul. 2018. Dasar Pemrograman Java. Yogyakarta: Andi Offset.
2. Nugroho, Adi. 2019. Struktur Data dengan Java. Bandung: Informatika.
3. Sedgewick, Robert dan Kevin Wayne. 2011. Algorithms Fourth Edition. Boston: AddisonWesley.
4. Oracle Corporation. 2026. Java Documentation. <https://docs.oracle.com/javase/>
5. Weiss, Mark Allen. 2014. Data Structures and Algorithm Analysis in Java. United States: Pearson Education.